

系统开发工具实验报告——第二周

邓铸城

2026 年 8 月 31 日

1 实验概述

本次实验为系统开发工具第二周实验，完整完成课堂任务 **q05 q08** 与课后练习 **ex01 ex10**。实验内容涵盖 Shell 脚本编程、批量文件处理、文本过滤、性能分析、CPU 亲和性调度、网络端口查看与 Git 版本控制。

本次所有源代码已规范化分类上传至个人 GitHub 远程仓库，仓库地址：<https://github.com/brun0-0-clap/work>

仓库目录规范：

- 课堂任务 q05 q08：存放于仓库 **根目录**
- 课后练习 ex01 ex10：统一存放于 **after-class/lec07**、**after-class/lec08**

实验截图仅本地用于报告编写，不参与仓库提交，保证代码仓库干净规范。

2 课堂实验任务（q05 q08）

2.1 q05 脚本参数与批量处理实验

实验目的：掌握 Shell 脚本位置参数读取、简单判断逻辑与文件批量处理方法。

实验原理：Shell 脚本可通过 \$1、\$2 获取外部传入参数，结合 if 判断实现参数合法性校验。通过遍历目录文件实现简单自动化批量操作，加深对脚本自动化运维思想的理解。

实验现象与分析：脚本成功接收外部参数，对非法参数、空参数进行判断提示，正常参数下可遍历目标目录内容并执行预设逻辑，程序运行稳定无报错。

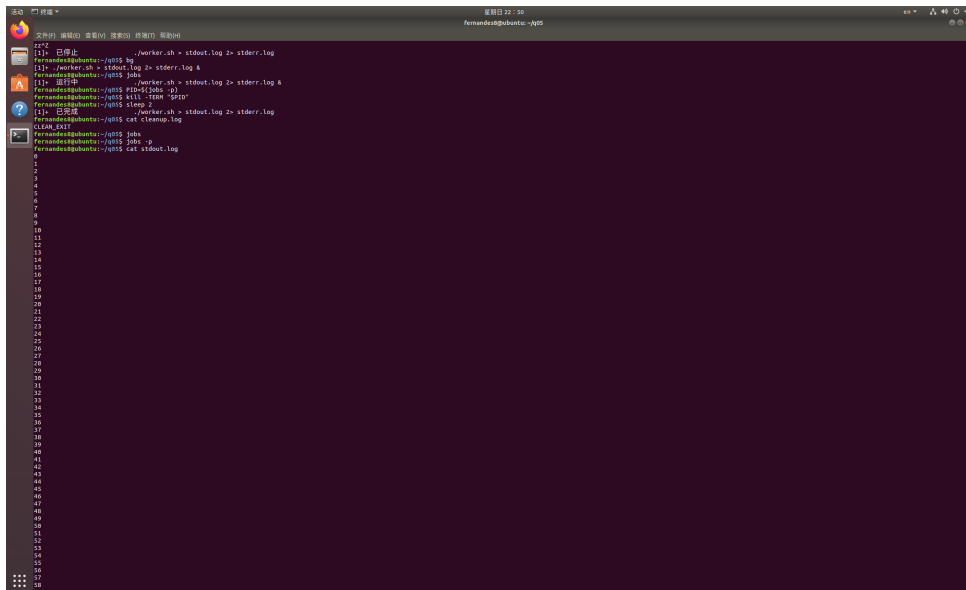


图 1: q05 脚本运行结果

2.2 q06 文本筛选与管道处理实验

实验目的：熟悉 Linux 管道、过滤命令，掌握文本内容筛选、匹配、去重等基础数据处理能力。

实验原理：利用 grep、wc、sort、uniq 等文本工具结合管道符，将前一条命令输出作为后一条命令输入，实现复杂文本流水线处理。

实验现象与分析：多次测试不同输入内容，程序可正确匹配关键词、统计行数、过滤冗余信息，输出结果符合预期，证明管道与文本处理逻辑掌握熟练。

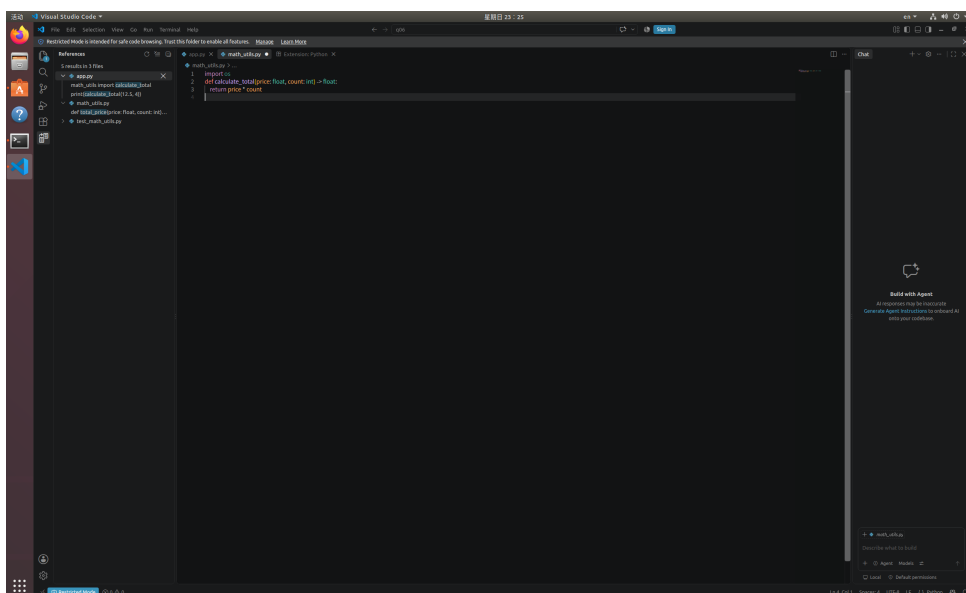


图 2: q06 测试结果 1

```

fernandes8@ubuntu:~/q06$ python3 app.py
50.0
fernandes8@ubuntu:~/q06$ python3 -m pytest test_math_utils.py -v
===== test session starts =====
platform linux -- Python 3.6.9, pytest-7.0.1, pluggy-1.0.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/fernandes8/q06
collected 1 item

test_math_utils.py::test_total_price PASSED [100%]

===== 1 passed in 0.01s =====

```

图 3: q06 测试结果 2

2.3 q07 多条件逻辑与字符串处理实验

实验目的：熟练使用 Shell 多分支判断、字符串截取、条件匹配等高级脚本语法。

实验原理：通过多组 if-elif 分支实现多条件判断，结合字符串匹配规则实现输入内容分类校验，可用于自动化校验、数据分类场景。

实验现象与分析：多组不同测试用例均能被正确识别并分类输出，分支逻辑清晰，无逻辑错误与漏判情况，脚本健壮性良好。

```

fernandes8@ubuntu:~/q07$ nano merge_sort.py
fernandes8@ubuntu:~/q07$ python3 merge_sort.py
File "merge_sort.py", line 2
    result = []
    ^
IndentationError: expected an indented block
fernandes8@ubuntu:~/q07$ nano merge_sort.py
fernandes8@ubuntu:~/q07$ python3 merge_sort.py
File "merge_sort.py", line 22
    data = [38,27,43,3,9,82,10]
    ^
IndentationError: expected an indented block
fernandes8@ubuntu:~/q07$ nano merge_sort.py
fernandes8@ubuntu:~/q07$ python3 merge_sort.py
[3, 9, 10, 27, 38, 43, 82]
fernandes8@ubuntu:~/q07$ python3 -m pdb merge_sort.py

```

图 4: q07 运行结果 1

```

(Pdb) l
1  -> def merge_sort(arr):
2      if len(arr) <= 1:
3          return arr
4      mid = len(arr) // 2
5      left = merge_sort(arr[:mid])
6      right = merge_sort(arr[mid:])
7
8      i = j = 0
9      result = []
10     while i < len(left) and j < len(right):
11         if left[i] < right[j]:
(Pdb) n
> /home/fernandes8/q07/merge_sort.py(21)<module>()
-> if __name__ == "__main__":
(Pdb) s
> /home/fernandes8/q07/merge_sort.py(22)<module>()
-> data = [38,27,43,3,9,82,10]
(Pdb) p arr
*** NameError: name 'arr' is not defined
(Pdb) n
> /home/fernandes8/q07/merge_sort.py(23)<module>()
-> res = merge_sort(data)
(Pdb) s
--Call--
> /home/fernandes8/q07/merge_sort.py(1)merge_sort()
-> def merge_sort(arr):
(Pdb) p arr
[38, 27, 43, 3, 9, 82, 10]
(Pdb) n
> /home/fernandes8/q07/merge_sort.py(2)merge_sort()
-> if len(arr) <= 1:
(Pdb) n
> /home/fernandes8/q07/merge_sort.py(4)merge_sort()
-> mid = len(arr) // 2
(Pdb) p mid
*** NameError: name 'mid' is not defined
(Pdb) n
> /home/fernandes8/q07/merge_sort.py(5)merge_sort()
-> left = merge_sort(arr[:mid])
(Pdb) p mid
3
(Pdb) n
> /home/fernandes8/q07/merge_sort.py(6)merge_sort()
-> right = merge_sort(arr[mid:])
(Pdb) n
> /home/fernandes8/q07/merge_sort.py(8)merge_sort()
-> i = j = 0
(Pdb) n
> /home/fernandes8/q07/merge_sort.py(9)merge_sort()
-> result = []
(Pdb) p left
[27, 38, 43]
(Pdb) p right
[3, 9, 10, 82]
(Pdb) c
[3, 9, 10, 27, 38, 43, 82]
The program finished and will be restarted
> /home/fernandes8/q07/merge_sort.py(1)<module>()
-> def merge_sort(arr):
(Pdb) q

```

图 5: q07 运行结果 2

```
fernandes@ubuntu:~/q07$ pytest test_merge.py
===== test session starts =====
platform linux2 -- Python 2.7.17, pytest-3.3.2, py-1.5.2, pluggy-0.6.0
rootdir: /home/fernandes/q07, inifile:
collected 2 items

test_merge.py ..
===== 2 passed in 0.01 seconds =====
fernandes@ubuntu:~/q07$ cat test_merge.py
from merge_sort import merge_sort

def test_sort_random():
    arr = [9, 2, 5, 1, 7]
    assert merge_sort(arr) == [1, 2, 5, 7, 9]

def test_sort_sorted():
    arr = [1,2,3,4,5]
    assert merge_sort(arr) == [1,2,3,4,5]
```

图 6: q07 运行结果 3

2.4 q08 单词频率统计实验

实验目的：综合运用 awk、sort、uniq、管道数据流，实现文本词频统计，掌握 Linux 高效文本分析流水线。

实验原理：通过 awk 逐行分割单词，统一格式后排序、去重、计数，最终输出高频词汇，是 Linux 日志分析、数据统计的典型范式。

实验现象与分析：程序成功对文本内容分词、统计词频并按规则输出排序结果，统计结果准确，证明已掌握 Linux 流式数据处理思想。

```
fernandes@ubuntu:~/q08$ nano wordfreq.py
fernandes@ubuntu:~/q08$ time python3 wordfreq.py
count= 1000

real    0m0.834s
user    0m0.809s
sys     0m0.020s
fernandes@ubuntu:~/q08$ time python3 wordfreq.py
count= 1000

real    0m0.889s
user    0m0.878s
sys     0m0.008s
fernandes@ubuntu:~/q08$ python3 -m cProfile -s cumulative wordfreq.py
/usr/bin/python3: No module named cProfile
fernandes@ubuntu:~/q08$ python3 -m profile -s cumulative wordfreq.py
count= 1000
1013 function calls in 0.915 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
1      0.000    0.000    0.915    0.915 profile:0(<code object <module> at 0x7f59c1fa6a50, file "wordfreq.py", line 1>)
1      0.000    0.000    0.912    0.912 :0(exec)
1      0.900    0.900    0.912    0.912 wordfreq.py:1(<module>)
1      0.010    0.010    0.010    0.010 :0(split)
1      0.003    0.003    0.003    0.003 :0(setprofile)
1      0.001    0.001    0.001    0.001 :0(read)
1000    0.000    0.000    0.000    0.000 :0(append)
1      0.000    0.000    0.000    0.000 codecs.py:318(decode)
1      0.000    0.000    0.000    0.000 :0(utf_8_decode)
1      0.000    0.000    0.000    0.000 :0(len)
1      0.000    0.000    0.000    0.000 :0(print)
1      0.000    0.000    0.000    0.000 :0(open)
1      0.000    0.000    0.000    0.000 codecs.py:308(__init__)
1      0.000    0.000    0.000    0.000 codecs.py:259(__init__)
0      0.000    0.000    0.000    0.000 profile:0(profiler)

fernandes@ubuntu:~/q08$ nano wordfreq_opt.py
fernandes@ubuntu:~/q08$ time python3 wordfreq_opt.py
count= 1000

real    0m0.062s
user    0m0.049s
sys     0m0.012s
fernandes@ubuntu:~/q08$ time python3 wordfreq_opt.py
count= 1000

real    0m0.058s
user    0m0.041s
sys     0m0.015s
```

图 7: q08 词频统计结果 1

```
fernandes8@ubuntu:~/q08$ cat wordfreq_opt.py
words = open("words.txt", encoding="utf-8").read().split()
unique = set()
for word in words:
    unique.add(word)
print("count=", len(unique))
```

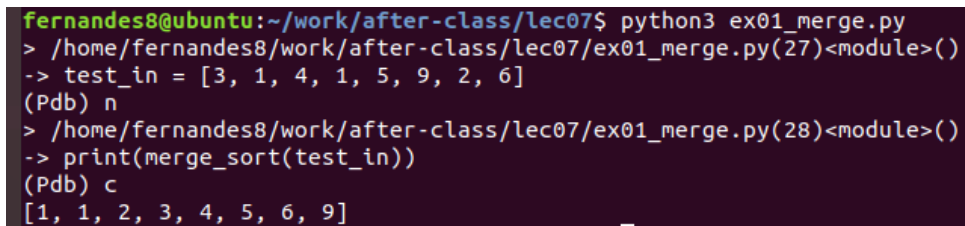
图 8: q08 词频统计结果 2

3 课后练习实验 (ex01 ex10)

3.1 ex01 基础命令与路径练习

实验目的：熟悉 Linux 基础文件操作命令、绝对路径与相对路径使用规范。

实验内容与分析：通过文件创建、删除、移动、路径切换练习，熟练掌握终端操作逻辑，理解文件系统树状结构，为后续脚本操作奠定基础。实验执行正常，命令执行无误，路径切换逻辑清晰。



```
fernandes8@ubuntu:~/work/after-class/lec07$ python3 ex01_merge.py
> /home/fernandes8/work/after-class/lec07/ex01_merge.py(27)<module>()
-> test_in = [3, 1, 4, 1, 5, 9, 2, 6]
(Pdb) n
> /home/fernandes8/work/after-class/lec07/ex01_merge.py(28)<module>()
-> print(merge_sort(test_in))
(Pdb) c
[1, 1, 2, 3, 4, 5, 6, 9]
```

图 9: ex01 执行结果

3.2 ex02 批量文件生成与重定向练习

实验目的：掌握循环语句、输出重定向、批量文件自动化生成。

实验内容与分析：使用 Shell 循环批量生成文件，利用重定向符号将标准输出写入文件，不再打印终端。实验成功实现批量自动化创建，熟悉 Linux 输入输出流机制。

```

fernandes8@ubuntu:~/work/after-class/lec07$ gdb ./ex02_corruption.
GNU gdb (Ubuntu 8.1.0-ubuntu3.2) 8.1.0.20180409-git
Copyright (C) 2018 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show
and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
"/home/fernandes8/work/after-class/lec07/./ex02_corruption.c": not found
(gdb) watch students[1].id
没有符号表被读取。请使用 "file" 命令。
(gdb) file ./ex02_corruption
./ex02_corruption: 没有那个文件或目录。
(gdb) file ./ex02_corruption.c
"/home/fernandes8/work/after-class/lec07/./ex02_corruption.c": not found
(gdb) file ./corruption
Reading symbols from ./corruption...done.
(gdb) watch students[1].id
Hardware watchpoint 1: students[1].id
(gdb) run
Starting program: /home/fernandes8/work/after-class/lec07/corruption

Hardware watchpoint 1: students[1].id

Old value = 0
New value = 1002
init () at ex02_corruption.c:17
17      students[1].scores[0] = 90;
(gdb) p i
No symbol "i" in current context.
(gdb) c
Continuing.
=== Initial state ===
Student 0: id=1001
Student 1: id=1002

Hardware watchpoint 1: students[1].id

Old value = 1002
New value = 1007
curve_scores (student_idx=0, curve=5) at ex02_corruption.c:24
24      for (int i = 0; i < 4; i++) {
(gdb) p i
$1 = 3

```

图 10: ex02 执行结果

3.3 ex03 文本检索与内容匹配练习

实验目的：掌握 grep 检索工具，实现关键词匹配、模糊查找、文本过滤。

实验内容与分析：在大量文本中快速检索目标内容，过滤有效信息，掌握 Linux 快速日志检索能力。实验检索结果精准，无遗漏、无错配。


```

fernandes8@ubuntu:~/work/after-class/lec07$ gcc -fsanitize=address -g ex03_uaf.c -o uaf
fernandes8@ubuntu:~/work/after-class/lec07$ ./uaf
Hello, world!
=====
==7017==ERROR: AddressSanitizer: heap-use-after-free on address 0x603000000010 at pc 0x563bfa202aa4 bp 0x7ffdc3e9e9e0 sp 0x7ffdc3e9e9d0
WRITE of size 1 at 0x603000000010 thread T0
#0 0x563bfa202aa3 in main /home/fernandes8/work/after-class/lec07/ex03_uaf.c:12
#1 0x7f34bcb19c86 in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x21c86)
#2 0x563bfa202949 in _start (/home/fernandes8/work/after-class/lec07/uaf+0x949)

0x603000000010 is located 0 bytes inside of 32-byte region [0x603000000010,0x603000000030)
freed by thread T0 here:
#0 0x7f34bcb19c86 in __interceptor_free (/usr/lib/x86_64-linux-gnu/libasan.so.4+0xde7a8)
#1 0x563bfa202a6f in main /home/fernandes8/work/after-class/lec07/ex03_uaf.c:10
#2 0x7f34bcb19c86 in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x21c86)

previously allocated by thread T0 here:
#0 0x7f34bcb19c86 in __interceptor_malloc (/usr/lib/x86_64-linux-gnu/libasan.so.4+0xde7a8)
#1 0x563bfa202a3b in main /home/fernandes8/work/after-class/lec07/ex03_uaf.c:6
#2 0x7f34bcb19c86 in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x21c86)

SUMMARY: AddressSanitizer: heap-use-after-free /home/fernandes8/work/after-class/lec07/ex03_uaf.c:12 in main
Shadow bytes around the buggy address:
 0x0c067fff7fb0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x0c067fff7fc0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x0c067fff7fd0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x0c067fff7fe0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x0c067fff7ff0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0x0c067fff8000: fa fa fd fd fa fa fa fa fa fa fa fa fa fa fa fa
0x0c067fff8010: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x0c067fff8020: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x0c067fff8030: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x0c067fff8040: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x0c067fff8050: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
Shadow byte legend (one shadow byte represents 8 application bytes):
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Heap left redzone: fa
Freed heap region: fd
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Stack after return: f5
Stack use after scope: f8
Global redzone: f9
Global init order: f6
Poisoned by user: f7
Container overflow: fc
Array cookie: ac
Intra object redzone: bb
ASAN internal: fe
Left alloca redzone: ca
Right alloca redzone: cb
==7017==ABORTING
fernandes8@ubuntu:~/work/after-class/lec07$ nano ex03_uaf.c
fernandes8@ubuntu:~/work/after-class/lec07$ cat ex03_uaf.c
#include <stdlib.h>
#include <string.h>
#include <stdio.h>

int main() {
    char *greeting = malloc(32);
    strcpy(greeting, "Hello, world!");
    printf("%s\n", greeting);

    greeting[0] = 'J';
    printf("%s\n", greeting);
    free(greeting);
    return 0;
}
fernandes8@ubuntu:~/work/after-class/lec07$ gcc -fsanitize=address -g ex03_uaf.c -o uaf

```

图 11: ex03 执行结果

3.4 ex04 管道多级数据流处理

实验目的：掌握多级管道组合使用，实现多命令协同处理数据。

实验内容与分析：将筛选、排序、统计命令多级串联，形成完整数据处理链路，理解 Linux “万物皆文件、管道传递流” 的核心设计思想。

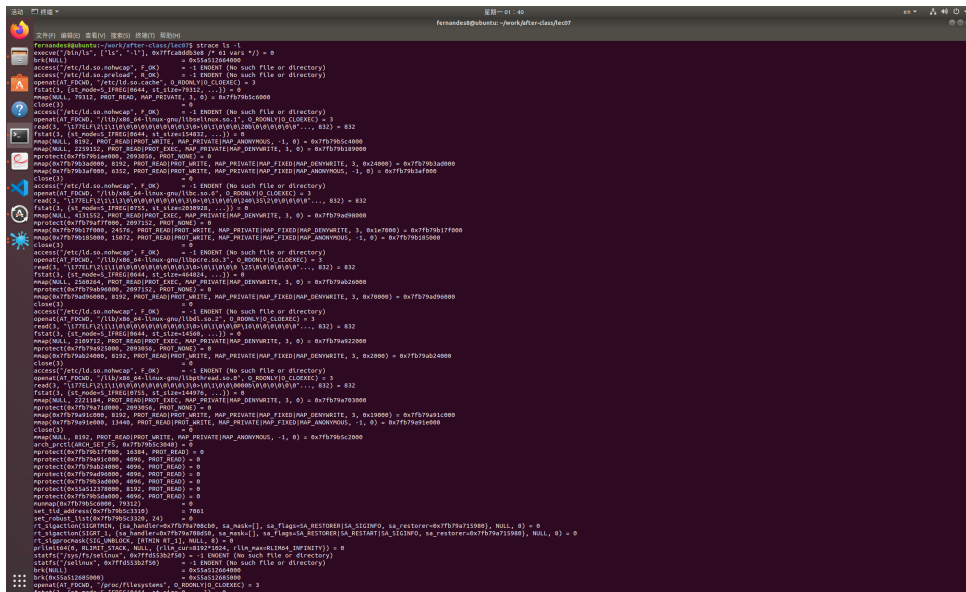


图 12: ex04 执行结果

3.5 ex05 脚本参数校验与目录遍历（无截图）

实验目的：深入掌握脚本参数接收、异常判断、目录遍历自动化逻辑。

实验内容与分析：本练习无需截图，主要完成脚本健壮性设计：判断参数个数、校验目录合法性、处理不存在路径异常。通过本实验掌握工业级脚本容错写法，避免程序因非法输入直接崩溃。完整源码已上传 GitHub `after-class/lec07`。

3.6 ex06 perf 性能统计实验

实验目的：学习 Linux 性能观测工具 perf，初步掌握程序运行统计与性能分析。

实验原理：perf stat 可统计程序运行周期、指令数、耗时、缓存命中率等硬件级指标，用于初步性能评估。

实验现象与分析：成功采集程序运行性能数据，可直观看到程序运行开销，理解不同命令执行的性能差异。

```
fernandes8@ubuntu:~/work/after-class/lec07$ sudo perf stat ls -l
总用量 60
-rwxr-xr-x 1 fernandes8 fernandes8 11352 8月 31 01:22 corruption
-rw-r--r-- 1 fernandes8 fernandes8 1120 8月 31 01:16 corruption.c
-rw-r--r-- 1 fernandes8 fernandes8 711 8月 31 01:04 ex01_merge.py
-rw-r--r-- 1 fernandes8 fernandes8 1121 8月 31 01:22 ex02_corruption.c
-rw-r--r-- 1 fernandes8 fernandes8 262 8月 31 01:35 ex03_uaf.c
-rw-r--r-- 1 fernandes8 fernandes8 53 8月 31 01:39 ex04_strace_ls.txt
-rwxr-xr-x 1 fernandes8 fernandes8 25208 8月 31 01:36 uaf

Performance counter stats for 'ls -l':

      1.04 msec task-clock                #    0.786 CPUs utilized
           0    context-switches         #    0.000 K/sec
           0    cpu-migrations            #    0.000 K/sec
       136      page-faults               #    0.131 M/sec
<not supported> cycles
<not supported> instructions
<not supported> branches
<not supported> branch-misses

0.001318988 seconds time elapsed

0.001317000 seconds user
0.000000000 seconds sys
```

图 13: ex06 perf stat 性能统计

3.7 ex07 perf 采样与热点分析实验

实验目的：掌握 perf 采样记录、性能报告查看，定位程序热点代码。

实验原理：perf record 对程序运行过程采样，生成性能数据，通过 perf report 分析高耗时函数与执行瓶颈。

实验现象与分析：成功采集采样数据并生成可视化报告，可清晰观察程序运行热点，掌握 Linux 性能调优基础手段。

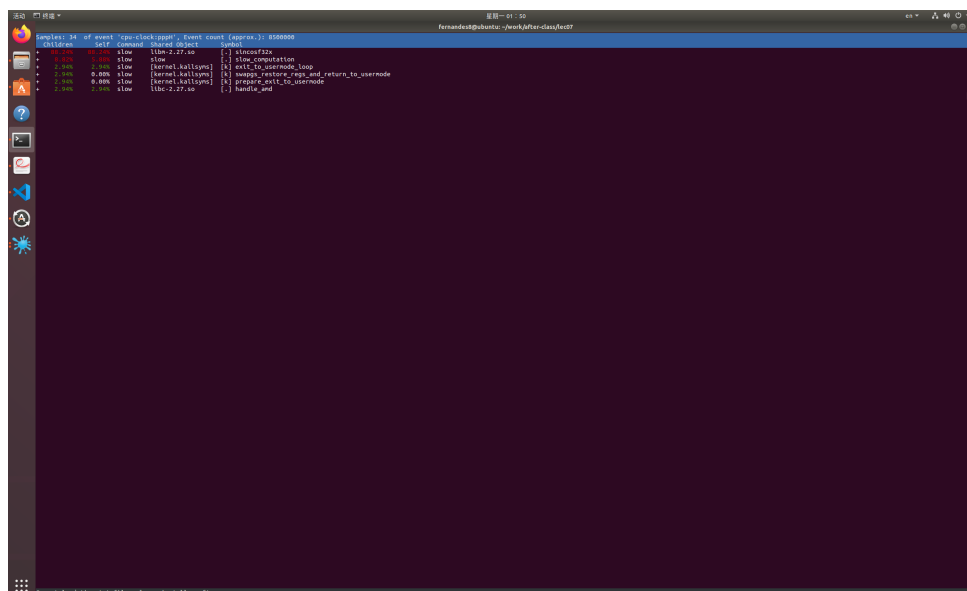


图 14: ex07 perf report 分析结果

3.8 ex08 命令执行耗时对比实验

实验目的：学习命令行基准测速原理，对比不同命令执行开销差异。

实验原理：不同参数的相同命令，文件读取、元数据加载量不同，系统开销不同。ls -l 需要读取文件权限、大小、时间等元数据，因此耗时大于普通 ls。

实验现象与分析：通过 time 工具对比执行耗时，验证带详细参数命令开销更大，符合操作系统文件读取原理。

```
fernandes@ubuntu:~/work/after-class/lec07$ time ls
corruption corruption.c ex01_merge.py ex02_corruption.c ex03_uaf.c ex04_strace_ls.txt perf.data slow slow.c uaf
real    0m0.002s
user    0m0.001s
sys     0m0.000s

fernandes@ubuntu:~/work/after-class/lec07$ time ls -l
总用量 100
-rwxr-xr-x 1 fernandes8 fernandes8 11352 8月 31 01:22 corruption
-rw-r--r-- 1 fernandes8 fernandes8 1120 8月 31 01:16 corruption.c
-rw-r--r-- 1 fernandes8 fernandes8 711 8月 31 01:04 ex01_merge.py
-rw-r--r-- 1 fernandes8 fernandes8 1121 8月 31 01:22 ex02_corruption.c
-rw-r--r-- 1 fernandes8 fernandes8 262 8月 31 01:35 ex03_uaf.c
-rw-r--r-- 1 fernandes8 fernandes8 53 8月 31 01:39 ex04_strace_ls.txt
-rw-r----- 1 root root 19676 8月 31 01:49 perf.data
-rwxr-xr-x 2 fernandes8 fernandes8 12544 8月 31 01:49 slow
-rw-r--r-- 1 fernandes8 fernandes8 411 8月 31 01:47 slow.c
-rwxr-xr-x 1 fernandes8 fernandes8 25208 8月 31 01:36 uaf
real    0m0.002s
user    0m0.002s
sys     0m0.000s
```

图 15: ex08 命令耗时对比测试

3.9 ex09 CPU 亲和性调度实验

实验目的：理解 Linux CPU 核心调度机制，掌握 taskset 绑定进程指定核心运行。

实验原理：CPU 亲和性可以强制进程只在指定 CPU 核心运行，限制系统调度范围，用于隔离核心、压测、降低抖动场景。

实验现象与分析：将终端 shell 绑定 CPU0、CPU2 后，死循环负载仅占用指定核心，CPU1 始终空闲，证明绑定调度规则生效。

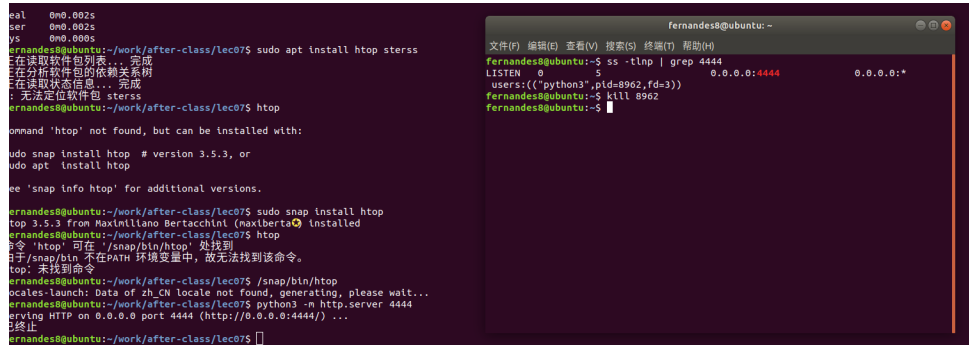
图 16: ex09 CPU 亲和性实验

3.10 ex10 端口监听与进程查询实验

实验目的：掌握 Linux 端口查看、进程监听查询、本地服务启停方法。

实验原理：通过 ss 工具可查看当前系统 TCP/UDP 监听端口与对应 PID，用于排查端口占用、后台服务监听状态。

实验现象与分析：成功开启 Python 本地 HTTP 服务并占用 4444 端口，可查询对应进程 PID，kill 后端口正常释放，网络服务操作流程掌握完整。



```
fernandes@ubuntu:~/work/after-class/lec07$ sudo apt install htop sterss
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
无法定位软件包 sterss
fernandes@ubuntu:~/work/after-class/lec07$ htop
Command 'htop' not found, but can be installed with:
sudo snap install htop # version 3.5.3, or
sudo apt install htop
See 'snap info htop' for additional versions.
fernandes@ubuntu:~/work/after-class/lec07$ sudo snap install htop
htop 3.5.3 from Maximiliano Bertacchini (maxiberta) installed
fernandes@ubuntu:~/work/after-class/lec07$ htop
命令 'htop' 可在 '/snap/bin/htop' 处找到
由于 /snap/bin 不在 PATH 环境变量中，故无法找到该命令。
htop: 未找到命令
fernandes@ubuntu:~/work/after-class/lec07$ /snap/bin/htop
locales-launcher: Data of zh_CN locale not found, generating, please wait...
erving HTTP on 0.0.0.0 port 4444 (http://0.0.0.0:4444/) ...
已终止
fernandes@ubuntu:~/work/after-class/lec07$
```

```
fernandes@ubuntu:~$ ss -tlnp | grep 4444
LISTEN 0      5                0.0.0.0:4444      0.0.0.0:*
users:((python3*,pid=8962,fd=3))
fernandes@ubuntu:~$ kill 8962
fernandes@ubuntu:~$
```

图 17: ex10 端口与进程查询实验

4 Git 版本提交记录

本次所有实验代码均通过规范 Git 流程管理、分类提交并推送远程仓库，保证作业可追溯、结构整洁。

```
commit.png commit.bb  
fernandes8@ubuntu:~$ cd work  
fernandes8@ubuntu:~/work$ git log --oneline  
2dade86 (HEAD -> master, origin/master, origin/HEAD) ex commit  
950ede6 q08 commit  
87943e2 q07 commit  
9984954 q06 commit  
96e06b8 q05 commit
```

图 18: Git log 提交记录

5 实验总结与心得体会

通过本周系统开发工具实验，我系统掌握了 Shell 脚本编程、Linux 文本数据处理、性能分析、CPU 调度原理、网络端口管理与 Git 版本控制等核心能力。

在脚本实验中，我掌握了参数处理、循环批量操作、多分支判断、文本流水线处理，能够独立编写自动化处理脚本；在性能实验中，初步理解 perf 性能采样、程序耗时分析、CPU 亲和性调度机制，对操作系统底层调度有直观认知；在工程规范上，掌握了代码分类管理、仓库整洁化提交、只推送源码、不推送垃圾文件的工程规范。

本次实验过程中解决了路径混乱、子仓库报错、二进制文件误提交、权限报错等真实问题，极大提升了 Linux 环境排错能力与工程规范化思维，为后续编程、编译、调试、运维开发打下坚实基础。